

```
1 import torch
2 import matplotlib.pyplot as plt
3 import random
4 #=====读入训练数据=====
5 #python是从当前项目的根目录开始找文件的，所以这里的路径是相对于当前目录的
6 words = open('Neural Network_Zero to hero/makemore/names.txt', "r").read().splitlines()
7
8 chars = sorted(list(set(''.join(words))))
9 stoi = {s: i+1 for i,s in enumerate(chars)}
10 stoi['.'] = 0
11 itos = {i:s for s,i in stoi.items()}
12
13
14
15
16 #=====设置上下文大小并且把words变成元组=====
17 block_size = 3#这个代表，我们当前模型预测下一个字符的时候，希望采用多少个字符作为context
18 X,Y = [],[]
19
20
21 for w in words:
22
23     #print(w)
24     #这里表示生成block_size次列表并且把它们链接起来，形成一个block_size大小的list
25     #并且列表里面的内容就是0.
26     #注意：如果是[对象]*3这样，声明出来的列表，三那么个指针都指向同一个对象
27     context = [0]*block_size
28
29     #把w名字加上一个'.'，然后作为被遍历的对象。比如emma就变成了emma.
30     for ch in w + '.':
31         ix = stoi[ch]#从字典里面，按照ch字符，来取出数字key
32         X.append(context)#把context对象放入X组，作为输入组
33         Y.append(ix)
34         #print(''.join(itos[i] for i in context),'----->',itos[ix])
35         context = context[1:]+[ix]
36         #这里相当于，创建按了一个新的context对象
37         #和原来的context对象不一样了。旧的context仍然储存在X列表里面，更改新的不影响旧的
```

```
38
39
40 # (1) 理解上面的ch字符在每一个字符串w内循环:
41 #     1.每一次内循环, 先找出当前ch, 对应的整数key, 记为ix
42 #     2.ix表示, 当前训练集希望预测出的ch, 也就是神经网络的output
43 #     3.context表示神经网络的输出。初始情况是[0, 0, 0], 也就是没有上下文。即“...”
44 #     因为首次循环显然是预测名字的首个字符, 当然没有前文context
45 #     4.然后重新创建context, context【1; 】表示丢掉context的第一个元素, 只取后两个元素
46 #     然后加上【ix】, 就表示当前被预测的ch, 变成预测再下一个ch的context了
47
48 # (1.5) 需要注意的是, 上面得到的label元组X, Y是有可能重复的。
49 #     比如处理“ababab”这个名字, 就可以出现多次(aba,b)
50
51
52
53 # (2) 什么是label?
54 #     1.label就是训练数据中的“正确答案”, 也叫目标值、ground truth。
55 #     2.在监督学习中, 每个样本由输入和label组成。模型的任务是: 给输入, 预测label。
56 #     3.在bigram/字符级语言模型中:
57 #     输入是当前字符 (或前几个字符), label是真实的下一个字符。
58 #     代码里X存输入, Y存label
59 #     4.计算loss时, 用模型预测的概率分布和真实label对比。训练就是调整参数, 让模型给真实label分配更高的概率
60
61
62
63 #这里最终得到两个tensor:
64 #     X的规格是[32,3], 因为我们暂时只选了五个名字来提取。只有32种可能输入, 每一种输入大小为3。毕竟context大小为3
65 #     Y的规格是[32], 每个元素都是X矩阵的一种输入对应的一种输出
66 X = torch.tensor(X)
67 Y = torch.tensor(Y)
68
69
70 # (3) 什么是embedding? -----看笔记
71
72 #=====开始设置神经网络的参数层们=====
73
74 C = torch.randn((27,2))
```

```
75 # C = torch.randn((27,2))
76 # 创建一个嵌入矩阵。
77 # 27表示字符表大小（26个字母加一个点号）。
78 # 2表示每个字符用2维向量表示。
79 # 这个矩阵是随机初始化的可训练参数。
80 # 训练后，每个字符会有一个学习到的向量表示。
81
82
83 embedding = C[X]
84 # embedding = C[X]
85 # X是输入张量，形状是(batch_size, 3)，每个元素是字符索引。
86 # C[X]做查表：把每个索引替换成C中对应的行向量。
87 # 结果embedding形状是(batch_size, 3, 2)。
88 # 意思是：每个样本有3个字符，每个字符变成一个2维向量。
89 # 这里的batch_size是32，因为我们之前只选择了五个名字，这五个名字只包括了32种不同的元组
90
91
92 W1 = torch.randn((6,100))
93 # 第一层线性层的权重矩阵。
94 # 输入维度是6：因为3个字符，每个字符2维，拼接后3*2=6。
95 # 输出维度是100：隐藏层神经元数量，可以自己设定。这里设定为100
96 # 这个矩阵是随机初始化的可训练参数。
97 B1 = torch.randn(100)
98 # B1 = torch.randn(100)
99 # 第一层线性层的偏置向量。
100 # 长度是100，对应隐藏层100个神经元。
101 # 每个神经元有一个偏置，随机初始化，可训练。
102
103
104 embedding = embedding.view(-1,6)
105 # 把embedding从形状(32,3,2)重塑为(32,6)。
106 # 32是batch_size，6是每个样本拼接后的向量长度。
107 # 这一步把3个字符的向量首尾拼接成一个6维向量。
108 # 这样每个样本得到一个6维输入，可以送入线性层做矩阵乘法。
109 # 这个操作的效率非常高
110 # 如果第一个参数是-1，那么Pytorch会自动推理出这里应该填什么。因为元素总量不变，一个是6
111
```

```
112
113 hidden_layer_1 = torch.tanh(embedding @ W1 + B1)
114 #(4)为什么这里需要用tanh处理一下?
115 # 问题核心: 如果只有线性层(矩阵乘法加偏置), 无论堆叠多少层, 整体仍然是线性的。
116 # 证明: 假设两层都没有激活函数——比如tanh
117 # 第一层输出:  $h = x @ W1 + B1$ 
118 # 第二层输出:  $y = h @ W2 + B2$ 
119 # 代入得:  $y = (x @ W1 + B1) @ W2 + B2$ 
120 #            $= x @ (W1 @ W2) + (B1 @ W2 + B2)$ 
121 # 令  $W' = W1 @ W2$ ,  $B' = B1 @ W2 + B2$ 
122 # 则  $y = x @ W' + B'$ , 这还是一个线性变换。
123 # 所以没有激活函数的话, 多层和单层在数学上等价, 堆叠没有意义。
124
125
126
127 #这里设置第二层参数层
128 #因为神经元隐藏层有一百个神经元, 所以W2的第一个维度是100
129 #因为输出层有27个神经元, 所以B2第一个维度也是27
130 W2 = torch.randn((100,27))
131 B2 = torch.randn(27)
132
133 output_layer = hidden_layer_1@W2+B2
134
135 counts = output_layer.exp()
136 prob = counts/counts.sum(1,keepdims=True)
137 #这里得到的概率矩阵:
138 # 规格是32*27。或者说batch_size*27
139 # 因为当前读入的5个单词, 一共生成了32个字符元组(有所重复)
140 # 27个是因为: 对于每一次输入的上下文信息, 也就是那三个输出, 我们希望获得下一个ch的概率分布
141 # 并且下一个ch有可能是27个中的一个。所以要求的prob当然是分布在这27个选项上的
142 # 所以总的来说, prob矩阵的第一行, 对应的就是第一个上下文, 在当前参数下, 输出的概率分布
143 # 也就是: (...----> e) 中, ...作为输入的时候, 下一个ch的概率分布
144
145
146 #我们下面这行代码做的事是:
147 # 获取对于第k个样本, 模型分配给"正确下一个字符"的概率。
148 # 前面的torch.arange(32)创建了0到31这个数组。表示当前训练集内0到31这32个context
```

```
149 # 后面的Y, 储存这32个context对应的正确输出
150 # 所以整行代码, 实际返回的就是一个一维长度32的向量。
151 # 向量的第k个元素, 表示第k个输入的context, 模型预测该context输出的正确率
152 # 我们希望这个向量内的值, 越大越好。越大表示模型预测的越准
153 target_probs = prob[torch.arange(len(X)), Y]
154 Loss = -target_probs.log().mean()
155
156
157 #可以采用torch内置的方法便捷计算Loss:
158 Loss = torch.nn.functional.cross_entropy(output_layer, Y)
159
160
161 #找出参数们:
162 parameters = [C, W1, B1, W2, B2]
163 for p in parameters:
164     p.requires_grad = True
165
166
167 #这部分开始进行学习:
168 def learning_in_full_batch():
169     for i in range(1000):
170         #forward propagate:
171         embedding = C[X]
172         embedding = embedding.view(-1, 6)
173         hidden_layer_1 = torch.tanh(embedding @ W1 + B1)
174         output_layer_1 = hidden_layer_1 @ W2 + B2
175         Loss = torch.nn.functional.cross_entropy(output_layer_1, Y)
176
177         #backward propagate
178         for p in parameters:
179             p.grad = None
180         Loss.backward()
181         for p in parameters:
182             p.data += -0.1 * p.grad
183
184         if(i%100 == 0):
185             print(Loss.data)
```

```
186
187     print('training_done')
188
189
190 #直接按照30000个名字来计算，开销太大了。
191 #我们每一次可以只取部分元组用于训练。原来一次训练要计算几十万个元组
192 #现在我们可以一次训练使用几十个----Andrej说这样就很不错了
193 def partial_learning():
194     for i in range(1000):
195         ix = torch.randint(0,X.shape[0],(32,))
196         #这行代码的功能：
197         #   randint是一个生成随机整数的函数。
198         #   0表示整数的下界----包括下界
199         #   X.shape返回的是一个张量，这里X是二维，所以返回的Tensor也是二维
200         #   我们取X.shape[0]，表示取出第一个分量，也就是一个整数。这个整数表示当前二元组总数量
201         #   我们用X.shape[0]作为第二个参数，表示生成随机整数的上界----不包括上界
202         #   传入(32,)表示希望生成的随机整数，填入一个形状为(32,)的张量
203
204         embedding = C[X[ix]]
205         # embedding = C[X]
206         # X是输入张量，形状是(batch_size, 3)，每个元素是字符索引。
207         # C (27,2) 表示给每一个字符，都映射到了一个2维张量
208         # C[X]做查表：把每个索引替换成C中对应的行向量。
209         # 结果embedding形状是(batch_size, 3, 2)。
210         # 意思是：每个样本有3个字符，每个字符变成一个2维向量。
211         # 但是特别地，我们这里传入C查表的是：X[ix]。
212         # 表示只选择，我们上一步内随机抽中的32个样本，组成 (32,3) 大小的张量，传入C查表
213
214
215     #高级检索的规则如下：
216     #   把索引张量里的每一个整数，替换成 C 中对应那一行（一整条向量）。
217     #   所以我们传入的二维索引X[ix]，传入C查表，实际上发生的就是：
218     #       X里面被选择到的元组-----也就是三个字符的元组-----里面的字符被逐个替换成C里面对应的向量
219     #   所以最终的规格是 (32,3,2)，表示：
220     #       被选择的32个元组，里面每一个元组有3个字符，每一个字符映射到一个二维张量
221     #       对应的意思就是：3个字符被映射成了数字。比如a对应1，那么就映射到C的第1个二维张量
222     #
```

```

223     #
224     # 看一个例子:
225     # C = torch.tensor([
226     #     [10, 11], # 第 0 行
227     #     [20, 21], # 第 1 行
228     #     [30, 31], # 第 2 行
229     # ]) # 形状 (3, 2)
230     #
231     # idx = torch.tensor([
232     #     [0, 2],
233     #     [1, 0],
234     # ]) # 形状 (2, 2)
235     #
236     # C[idx] # 形状 (2, 2, 2)
237     # tensor([[[10, 11], # idx[0][0]=0 → 取第 0 行
238     #          [30, 31]], # idx[0][1]=2 → 取第 2 行
239     #         [[20, 21], # idx[1][0]=1 → 取第 1 行
240     #          [10, 11]]]) # idx[1][1]=0 → 取第 0 行
241     #
242     hidden_layer = torch.tanh(embedding.view(-1, 6)@W1+B1)
243     output_layer = hidden_layer@W2+B2
244     Loss = torch.nn.functional.cross_entropy(output_layer, Y[ix])
245
246
247     #back porpagate
248     for p in parameters:
249         p.grad = None
250     Loss.backward()
251     for p in parameters:
252         p.data += -0.1*p.grad
253     if(i%100 ==0):
254         print(Loss.data)
255
256
257
258
259     #=====如何找出最好的learning rate?=====

```

```
260 #拿上面的函数改造一下。我们希望得到用不同的learning_rate,在训练过程中Loss的变化趋势
261 learning_rates = 10**torch.linspace(-3,0,1000)
262 #linspace(-3,0,1000)是线性等分函数, 返回一个在-3,0之间均匀分布, 长度为1000的一维张量。
263 #同时这个一维张量包括-3和0本身, -3是起点, 0是终点。内部元素构成等差数列
264 Loss_arrays = []
265
266 def find_learning_rate():
267     for i in range(1000):
268         #forward
269         ix = torch.randint(0,X.shape[0],(32,))
270         embedding = C[X[ix]]
271         hidden_layer = torch.tanh(embedding.view(-1,6)@W1+B1)
272         output_layer = hidden_layer@W2+B2
273         Loss = torch.nn.functional.cross_entropy(output_layer, Y[ix])
274         #back
275         for p in parameters:
276             p.grad = None
277         Loss.backward()
278         #learning
279         for p in parameters:
280             p.data += -learning_rates[i]*p.grad
281
282         Loss_arrays.append(Loss.item())
283
284         if(i%100 ==0):
285             print(Loss.data)
286     print('training_done')
287
288 #输出一个图: 横坐标是学习率, 纵坐标是当前学习率对应的Loss
289 plt.plot(learning_rates, Loss_arrays)
290 plt.xscale('log')
291 plt.xlabel('learning rate')
292 plt.ylabel('loss')
293 plt.show()
294
295
296 #为什么这样可以评测学习率?
```

```
297 #比如，中间存在一段Loss比较低，难道这部分对应的learning_rate很好吗？
298 # 因为方法内1000次训练内的每一次训练，采用的learning_rate都不同
299 # 这一段之所以低，是之前训练的总结果，而不只是当前learning_rate的结果。怎么能说是这段learning_rate好呢？
300 #解答：
301 # 它是在找"学习率的上界"，也就是大概到多大学率会崩。
302 # 从左往右看，学习率从小逐渐变大。
303 # 前段 loss 在下降，说明当前学习率还在有效范围内。
304 # 到某个点 loss 突然上升或剧烈震荡，说明学习率太大了，训练发散了。
305 # 这个"崩溃点"对应的学习率，就是一个粗略的上界。
306
307 #为什么用最低点？
308 #曲线最低点附近，表示"在崩溃之前，训练效果最好的那一段"。
309 #但正如你说的，那个低 loss 是前面所有小学习率累积的结果。
310 #所以最低点本身不能直接当作最佳学习率来用。
311 #它只是一个信号：崩溃点大概在哪儿。
312 #实际怎么用这个信号？
313 #取崩溃点对应学习率的十分之一左右，作为训练的初始学习率。
314 #比如崩溃点大约是  $10^{-1}$ ，那就用  $10^{-2}$  开始训练。
315 #这是一种启发式经验，不是严格的最优值。
316 #它只是帮你避开"学习率太大直接炸"和"学习率太小几乎不动"两个极端。
317
318
319
320
321 #=====防止过拟合=====
322 #数据集可以分成三类:80%是训练集，10%是开发集，10%是测试集
323 #如果全部都是训练集，那么容易过拟合。在解决训练集之外的问题的时候，Loss可能非常大
324
325 #上面其实已经定义过了这几个参数。这个方法是用来重新生成参数的
326 #hidden_nuerons表示隐藏层有多少个神经元
327 def initialize_parameters(C,W1,W2,B1,B2,parameters,hidden_nuerons):
328     W2 = torch.randn((hidden_nuerons,27))
329     B2 = torch.randn(27)
330     W1 = torch.randn((6,hidden_nuerons))
331     B1 = torch.randn(hidden_nuerons)
332     C = torch.randn((27,2))
333     parameters = [C,W1,B1,W2,B2]
```

```
334     return parameters
335
336
337 #定义一个方法，来把words里面的名字，分成几组数据集
338 def build_dataset(words):
339
340
341
342     random.seed(42)
343     random.shuffle(words)
344     n1 = int(0.8*len(words))
345     n2 = int(0.9*len(words))
346
347     #小工具：把一批名字转换成对应的X, Y元组
348     #注意 subset 是名字列表；这里复用了最开头建 X、Y 时的同一套逻辑
349     def make_xy(subset):
350         X, Y = [], []
351         for w in subset:
352             #每个名字开始时，上下文是 block_size 个 0（表示“还没有任何字符”）
353             context = [0]*block_size
354             for ch in w + '.':
355                 ix = stoi[ch]                # 当前字符，同时也是预测的正确标签
356                 X.append(context)            # 输入：前 block_size 个字符
357                 Y.append(ix)                # 标签：真实的下一个字符
358                 context = context[1:] + [ix] # 滑窗：丢掉最旧的字符，加入当前字符
359         return torch.tensor(X), torch.tensor(Y)
360
361     X_train, Y_train = make_xy(words[:n1])    # 80% 训练集：用来更新参数
362     X_dev, Y_dev = make_xy(words[n1:n2])    # 10% 开发集：用来调超参数、判断过拟合
363     X_test, Y_test = make_xy(words[n2:])    # 10% 测试集：最后评估，只碰一次
364
365     print('训练集:', X_train.shape, Y_train.shape)
366     print('开发集:', X_dev.shape, Y_dev.shape)
367     print('测试集:', X_test.shape, Y_test.shape)
368
369     return X_train, Y_train, X_dev, Y_dev, X_test, Y_test
370
```

```
371
372
373 #再定义一个learning方法，采用分类后的数据集
374 #并且分别打印训练集，开发集，测试集的Loss
375 def learning_with_splits():
376     #第一步：调用 build_dataset，拿到 训练/开发/测试 三份数据
377     X_train, Y_train, X_dev, Y_dev, X_test, Y_test = build_dataset(list(words))
378
379     #一个小工具：在任意一份数据上计算 Loss（只做前向，不更新参数）
380     #为什么单独抽出来？因为要评估三份数据，避免把同样的代码重复写三遍
381     def split_loss(X, Y):
382         embedding = C[X]
383         hidden_layer = torch.tanh(embedding.view(-1, 6) @ W1 + B1)
384         output_layer = hidden_layer @ W2 + B2
385         return torch.nn.functional.cross_entropy(output_layer, Y)
386
387     for i in range(20000):
388         #----- 前向传播（只取一个 mini-batch） -----
389         step_ix = torch.randint(0, X_train.shape[0], (32,))
390         embedding = C[X_train[step_ix]]
391         hidden_layer = torch.tanh(embedding.view(-1, 6) @ W1 + B1)
392         output_layer = hidden_layer @ W2 + B2
393         Loss = torch.nn.functional.cross_entropy(output_layer, Y_train[step_ix])
394
395         #----- 反向传播 -----
396         for p in parameters:
397             p.grad = None
398         Loss.backward()
399
400         #----- 更新参数 -----
401         for p in parameters:
402             p.data += -0.1 * p.grad
403
404         #----- 每隔 2000 步，评估三个数据集的 Loss -----
405         if i % 2000 == 0 or i == 19999:
406             with torch.no_grad():
407                 # 评估不需要梯度，省时间省内存
408                 train_loss = split_loss(X_train, Y_train)
```

```
408         dev_loss = split_loss(X_dev, Y_dev)
409         test_loss = split_loss(X_test, Y_test)
410         print(f'step {i:6d} | train {train_loss.item():.4f} | dev {dev_loss.item():.4f} | test {test_loss.item():.4f}')
411
412
413 #直接运行learning_with_splits()方法
414 #我们发现, 在hidden_neurons只有100的时候, 三部分测试集, 计算出的Loss差不多, 并且都高
415 #andrej说, 这种情况叫做欠拟合, 说明模型太弱了。为什么呢?
416 #为什么 train loss 高 = 欠拟合?
417 #   关键在于训练集的 loss 是模型"可以尽力去压低"的那个数。
418 #   训练时模型每一步都直接拿训练集的 loss 做梯度下降, 没有任何机制阻止它把训练 loss 压低
419 #   (只要它有足够的容量)。
420 #   所以如果连训练集都拟合不好, 说明问题出在模型本身的能力/训练程度上, 而不是"泛化"问题——这就是欠拟合。
421 #   反之, 如果训练 loss 能压到很低、但测试 loss 高
422 #   说明模型"有能力记住训练集", 只是没学到通用的规律——那是过拟合。
423 #那么如何解决?
424 #   -----最简单粗暴的方式就是增加隐藏层的神经元数量, 还有embedding层的维度
425 #
426 #神经网络本质:
427 #   其实数据集可以被抽象成一个函数。一堆输入, 对应一堆输出
428 #   但是数据集函数太抽象太复杂, 以至于我们很难找到一个解析解
429 #   所以我们希望找一个方式, 能近似的获取这个训练集函数的数值解
430 #   神经网络本身就是一个复杂函数, 我们希望调整这个函数的内在参数, 使它更好的拟合数据集函数
431 #   参数越多, 那么神经网络这个函数, 可以变形的形态就更多, 更复杂, 就有潜力去更好的拟合数据集函数
432 #   所以, embedding和W和B都是这个神经网络的参数, 可以让我们的神经网络表现更好
433 #   但是我们的训练集, 只是所有可能的输入的一部分。
434 #   只靠这些数据去训练, 很可能不够-----所以提升数据量, 是一个很简单粗暴优化性能的方法
435
436
437 #Andrej还认为可以调整的参数们:
438 #   1.Andrej在这里训练最好的表现是2.17的Loss函数
439 #   2.我们还可以通过增加参数量, embedding的维度, 学习率, context的字符数量等等,
440 #       每一次forward过程挑选的元组数量 (因为一次前向传播跑完几十万个元组太多了, 我们每次就随机选择一部分来前向传播)
441 #
442
```